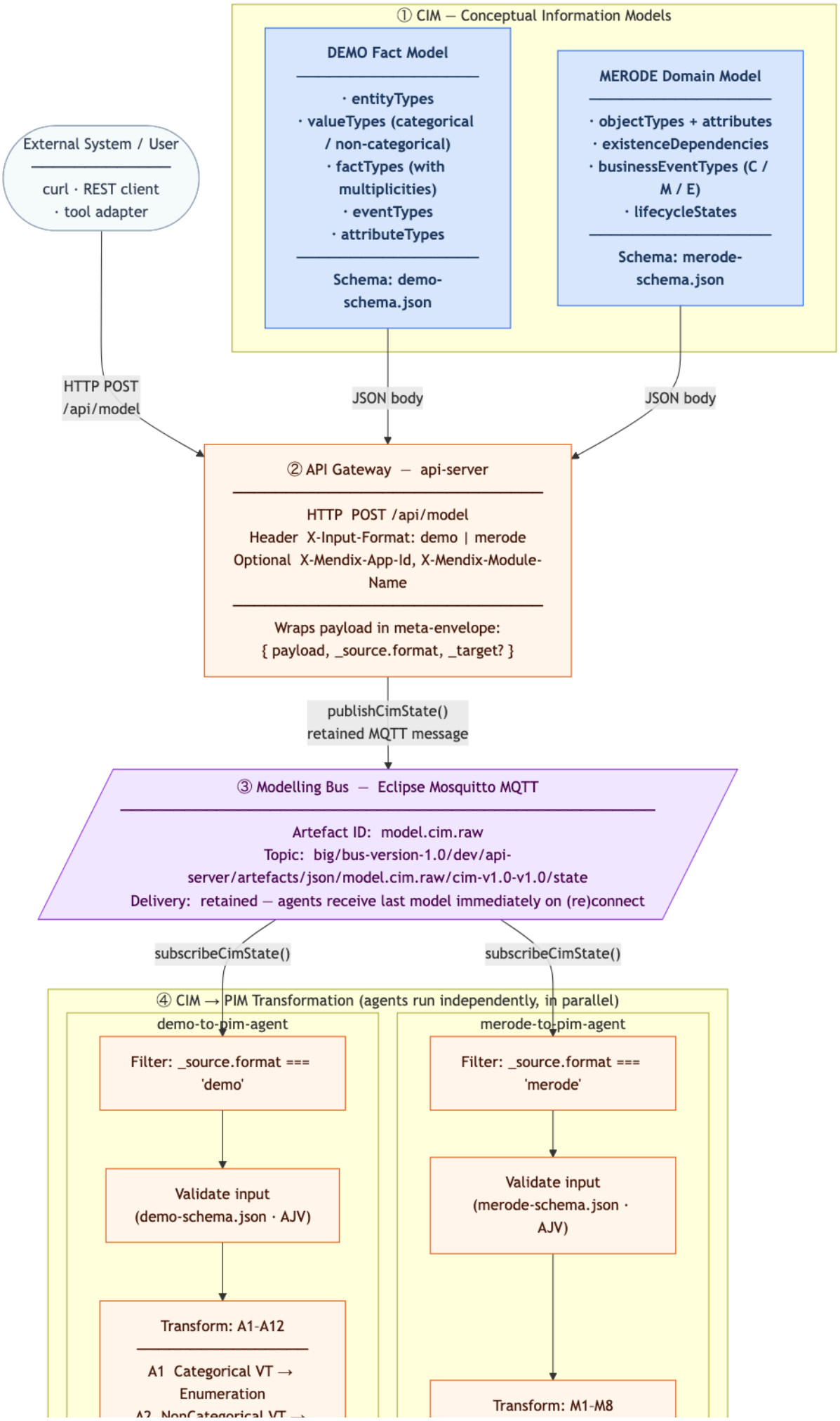
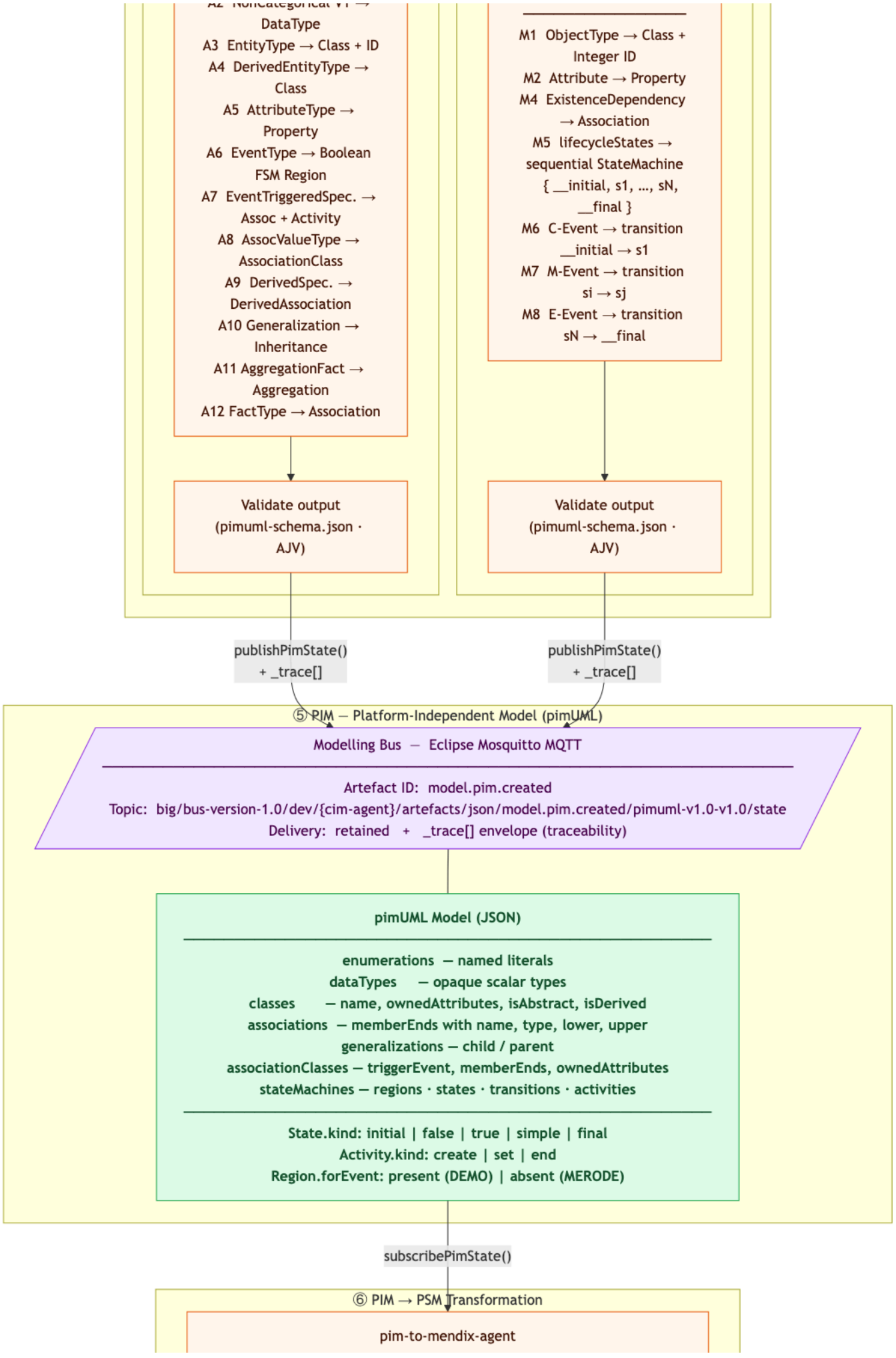
Architecture Overview: MDA Modelling Bus Prototype

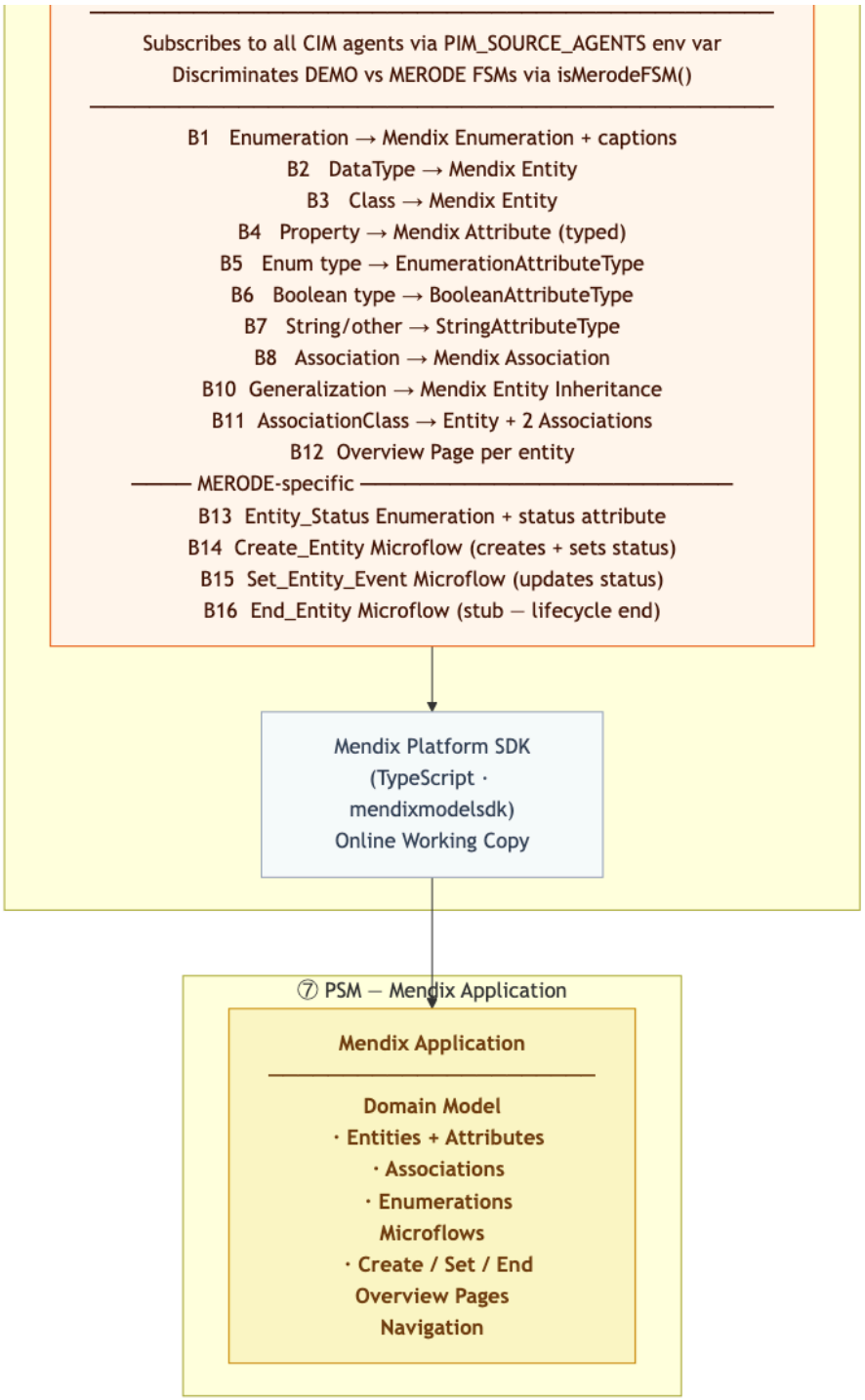
This document covers the technical architecture of the prototype built for my thesis. The system takes enterprise models (currently DEMO and MERODE) and transforms them into Mendix low-code apps, following the MDA abstraction hierarchy.

1. End-to-End Pipeline

The pipeline implements three MDA layers connected by an async message bus. Source models come in at the CIM level, get transformed into a shared PIM representation, and land as a Mendix app at the PSM level.

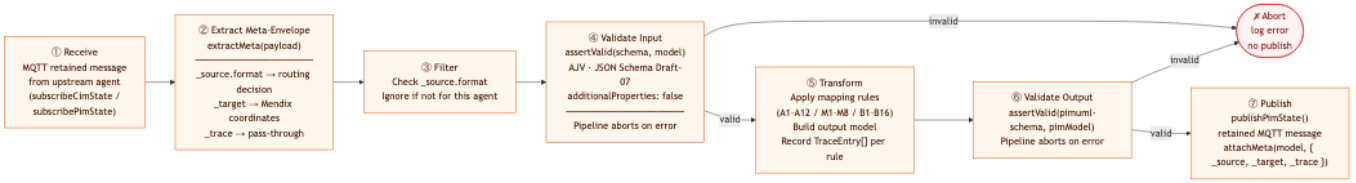

① CIM — Conceptual Information Models
DEMO Fact Model
· entityTypes
· valueTypes (categorical / non-categorical)
· factTypes (with multiplicities)
· eventTypes
· attributeTypes
Schema: demo-schema.json
MERODE Domain Model
· objectTypes + attributes
· existenceDependencies
· businessEventTypes (C / M / E)
· lifecycleStates
Schema: merode-schema.json
External System / User
curl · REST client
· tool adapter
HTTP POST /api/model
JSON body
JSON body
② API Gateway — api-server
HTTP POST /api/model
Header X-Input-Format: demo | merode
Optional X-Mendix-App-Id, X-Mendix-Module-Name
Wraps payload in meta-envelope:
{ payload, _source.format, _target? }
publishCimState()
retained MQTT message
③ Modelling Bus — Eclipse Mosquitto MQTT
Artefact ID: model.cim.raw
Topic: big/bus-version-1.0/dev/api-server/artefacts/json/model.cim.raw/cim-v1.0-v1.0/state
Delivery: retained — agents receive last model immediately on (re)connect
subscribeCimState()
subscribeCimState()
④ CIM → PIM Transformation (agents run independently, in parallel)
demo-to-pim-agent
Filter: _source.format === 'demo'
Validate input (demo-schema.json · AJV)
Transform: A1-A12
A1 Categorical VT → Enumeration
merode-to-pim-agent
Filter: _source.format === 'merode'
Validate input (merode-schema.json · AJV)
Transform: M1-M8


DataType
A3 EntityType → Class + ID
A4 DerivedEntityType → Class
A5 AttributeType → Property
A6 EventType → Boolean FSM Region
A7 EventTriggeredSpec. → Assoc + Activity
A8 AssocValueType → AssociationClass
A9 DerivedSpec. → DerivedAssociation
A10 Generalization → Inheritance
A11 AggregationFact → Aggregation
A12 FactType → Association
M1 ObjectType → Class + Integer ID
M2 Attribute → Property
M4 ExistenceDependency → Association
M5 lifecycleStates → sequential StateMachine { __initial, s1, …, sN, __final }
M6 C-Event → transition __initial → s1
M7 M-Event → transition si → sj
M8 E-Event → transition sN → __final
Validate output (pimuml-schema.json · AJV)
Validate output (pimuml-schema.json · AJV)
publishPimState() + _trace[]
publishPimState() + _trace[]
⑤ PIM — Platform-Independent Model (pimUML)
Modelling Bus — Eclipse Mosquitto MQTT
Artefact ID: model.pim.created
Topic: big/bus-version-1.0/dev/{cim-agent}/artefacts/json/model.pim.created/pimuml-v1.0-v1.0/state
Delivery: retained + _trace[] envelope (traceability)
pimUML Model (JSON)
enumerations — named literals
dataTypes — opaque scalar types
classes — name, ownedAttributes, isAbstract, isDerived
associations — memberEnds with name, type, lower, upper
generalizations — child / parent
associationClasses — triggerEvent, memberEnds, ownedAttributes
stateMachines — regions · states · transitions · activities
State.kind: initial | false | true | simple | final
Activity.kind: create | set | end
Region.forEvent: present (DEMO) | absent (MERODE)
subscribePimState()
⑥ PIM → PSM Transformation
pim-to-mendix-agent



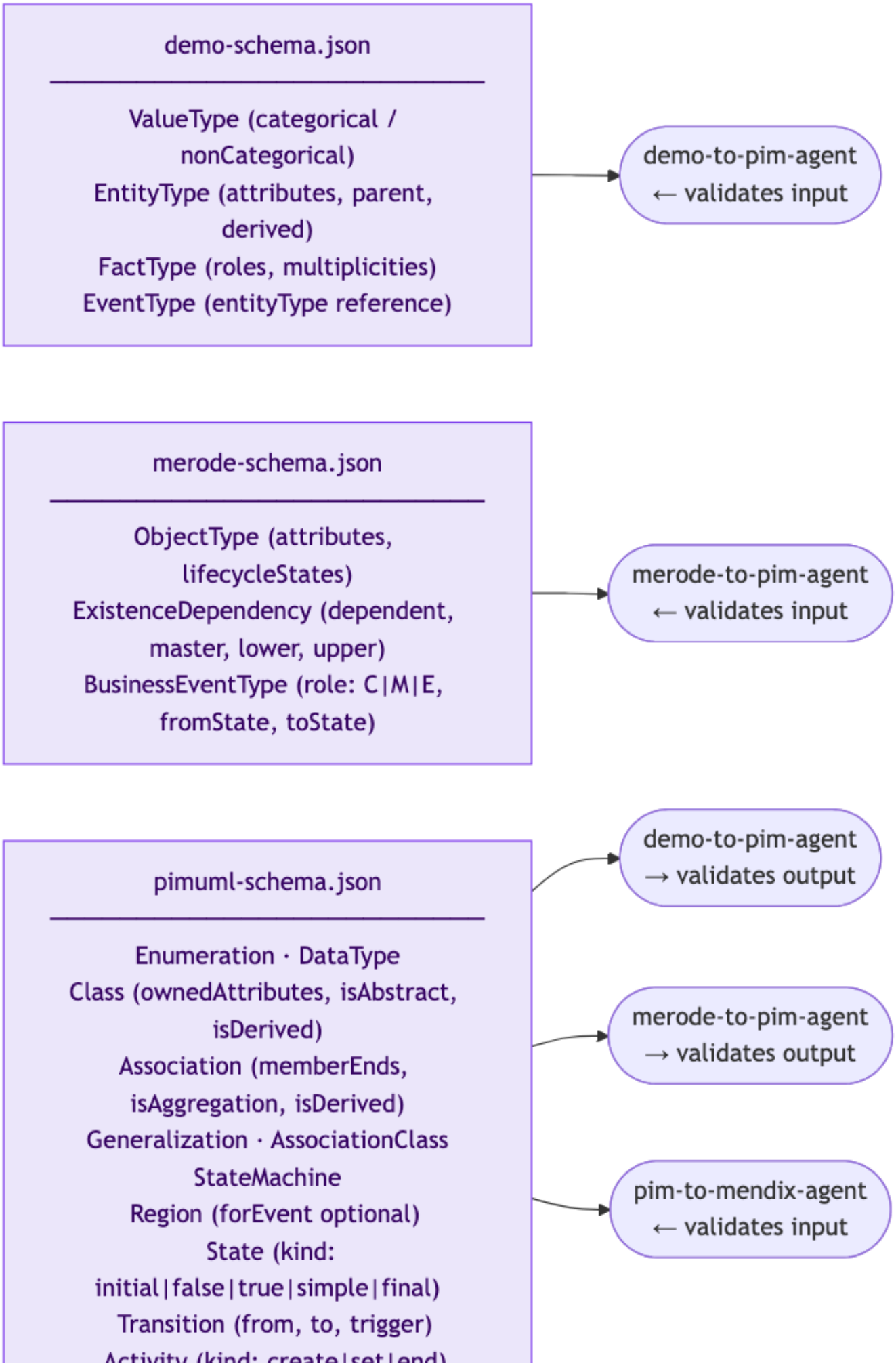
2. Internal Processing Pattern

Every agent, whether CIM-to-PIM or PIM-to-PSM, follows the same internal processing pattern. The shared infrastructure in `bus-client.ts` and `agent-utils.ts` enforces this structure.



3. Schema Hierarchy and Validation

Schema validation is a core concern. Every boundary in the pipeline is guarded by an AJV-compiled JSON Schema with `additionalProperties: false`. If validation fails the pipeline aborts without publishing anything.


demo-schema.json
ValueType (categorical / nonCategorical)
EntityType (attributes, parent, derived)
FactType (roles, multiplicities)
EventType (entityType reference)
demo-to-pim-agent
← validates input
merode-schema.json
ObjectType (attributes, lifecycleStates)
ExistenceDependency (dependent, master, lower, upper)
BusinessEventType (role: C|M|E, fromState, toState)
merode-to-pim-agent
← validates input
pimuml-schema.json
Enumeration · DataType
Class (ownedAttributes, isAbstract, isDerived)
Association (memberEnds, isAggregation, isDerived)
Generalization · AssociationClass
StateMachine
Region (forEvent optional)
State (kind: initial|false|true|simple|final)
Transition (from, to, trigger)
demo-to-pim-agent
→ validates output
merode-to-pim-agent
→ validates output
pim-to-mendix-agent
← validates input

Activity (kind: create|set|end)

The pimUML schema is the single shared contract between all CIM agents and the PSM agent. When MERODE was added the schema got extended additively (new `kind` enum values, `forEvent` made optional), so no existing DEMO-generated PIM document needed to change.

4. MQTT Topic Structure

The bus follows the BIG Modelling Bus naming convention (layer 1 of the Go reference implementation). Each agent writes into its own namespace; write topics are never shared.

```
{prefix}/{bus-version}/{environmentID}/{agentID}/artefacts/json/{artefactID}/{jso
```

Field	Value (this prototype)
`prefix`	`big`
`bus-version`	`bus-version-1.0`
`environmentID`	`dev`
`agentID`	`api-server`, `demo-to-pim-agent`, `merode-to-pim-agent`, `pim-to-mendix-agent`
`artefactID`	`model.cim.raw`, `model.pim.created`
`jsonVersion`	`cim-v1.0-v1.0`, `pimuml-v1.0-v1.0`
`state`	retained — newly connected agents immediately get the last known model

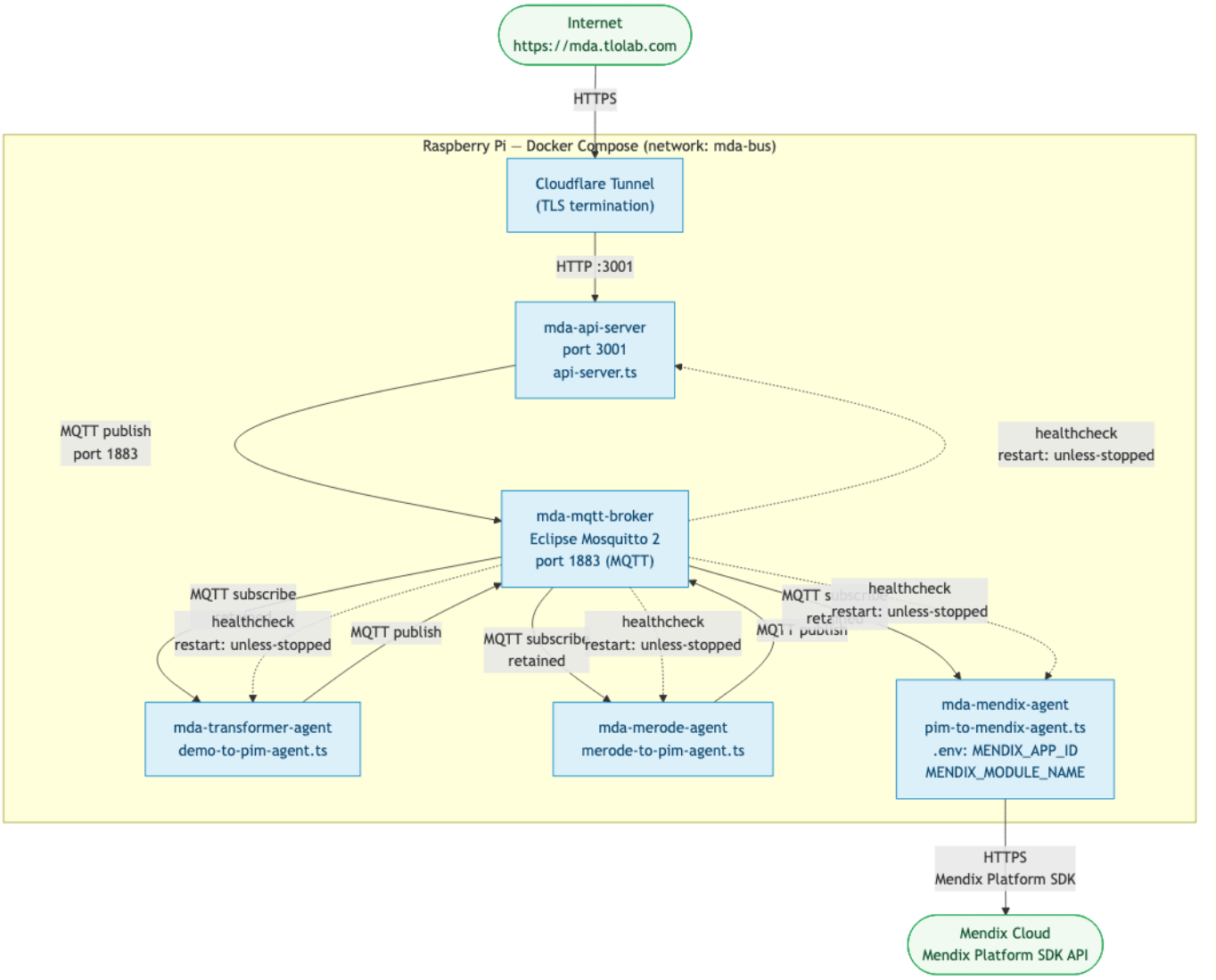
Example topics:

```
big/bus-version-1.0/dev/api-server/artefacts/json/model.cim.raw/cim-v1.0-v1.0/stat
big/bus-version-1.0/dev/demo-to-pim-agent/artefacts/json/model.pim.created/pimuml-
big/bus-version-1.0/dev/merode-to-pim-agent/artefacts/json/model.pim.created/pimu
```

The PSM agent subscribes to the PIM topics of both CIM agents. The set of upstream agents is controlled via the `PIM_PUBLISHER_AGENT_IDS` env var. Adding a third CIM agent only requires updating that variable, no code changes to the PSM agent.

5. Deployment

All components run as Docker containers on a Raspberry Pi. Public access goes through a Cloudflare tunnel that terminates TLS and forwards HTTPS traffic to the local API gateway.



CI/CD: a GitHub Actions workflow (`deploy-pi.yml`) deploys the stack automatically on every push to `main`. It SSHes into the Pi, pulls the latest image, and runs `docker compose up -d`.

6. Extensibility

The architecture is designed to absorb new CIM languages without touching existing components. The MERODE integration is the concrete proof of that.

Component	Change when adding MERODE
`demo-to-pim-agent.ts`	No changes
`bus-client.ts`	No changes
`agent-utils.ts`	No changes
`merode-to-pim-agent.ts`	New file (fully additive)

`merode-schema.json`	New file (fully additive)
`pimuml-schema.json`	Additive extension (new `kind` values, `forEvent` optional)
`types.ts`	New interfaces and small type extensions
`pim-to-mendix-agent.ts`	New B13-B16 rule blocks and `isMerodeFSM()` discriminator
`docker-compose.yml`	One new service block

A planned third-agent experiment (ER diagram language) predicts zero changes to existing files, because ER concepts map onto existing pimUML constructs (Classes, Associations) that B1-B12 already handle.

The extensibility boundary: adding a language with existing PIM concepts (classes, associations, attributes) only needs a new CIM agent. Adding a language with genuinely new PIM concepts (like MERODE's sequential FSMs) additionally needs a minimal additive schema extension and new PSM rules.

7. Meta-Envelope

Every message on the bus carries a meta-envelope alongside the model payload. The envelope is stripped before schema validation and re-attached afterwards, so `additionalProperties: false` isn't violated.

```
{
  ...model payload...,
  "_source": { "format": "demo" | "merode" },
  "_target": {
    "appId":      "<Mendix app UUID>",
    "moduleName": "<module name>",
    "branch":     "<branch>"
  },
  "_trace": [
    {
      "sourceType": "EntityType",
      "sourceName": "Rental",
      "targetType": "Class",
      "targetName": "Rental"
    },
    ...
  ]
}
```

The `_trace` array gives full end-to-end traceability: every rule application gets recorded as a `TraceEntry`. Entries with `targetType: "NotMapped"` explicitly document which source

concepts are out of scope (e.g. M9 propagation, derived OCL attributes).

8. Planned Real Tool Export Integration

Right now the prototype uses hand-crafted JSON fixtures as input. Wiring up real model exports from actual DEMO and MERODE tools is planned as a final step, after all rules are validated against the fixtures.

For MERODE the plan is to use Merlin (the reference tool from Snoeck). Merlin exports `.mxp` files; the open question is whether that format can feed directly into `merode-to-pim-agent` or whether an intermediate conversion step is needed first.

For DEMO the situation is less clear. simplified.engineering is a web-based DEMO modelling tool that in principle supports OFD exports. I need to check whether those exports actually carry the full structure of the A-rules (Bzwoski)

I probably need a parser for both.